



Cross-Game Transferability of Mouse-Dynamics Bot Detection

MSc Computer Science

Yu-Ching Chang

Supervised by Professor Panagiotis Andriotis

School of Computer Science

College of Engineering and Physical Sciences

University of Birmingham

2025-26

Honour Code

I certify that this project is my own work. The research question, experimental design, feature design, and the analysis and interpretation of results are my own. Code development and debugging were assisted by Claude Opus 4.8 and Cursor. All code was reviewed, tested and integrated by me. The report structure and initial drafts are mine; final editing was assisted by Claude for clarity and consistency. All outputs have been reviewed and edited to accurately reflect my work.

Abstract

This study investigates whether behaviour-based bot detectors can transfer across games. Existing mouse-behaviour-based methods are typically developed on a game-by-game basis, creating a cold-start problem when a new game launches without available cheat samples for training. If the behavioural differences between humans and bots are shared across games, a detector trained on an existing game could be applied directly to a new one; however, this possibility has not been systematically evaluated. Three public game datasets, comprising two first-person shooters (FPS) and one multiplayer online battle arena (MOBA), are unified into (dx, dy, t) displacement sequences. Four synthetic attacks of increasing realism, rule-based, parametric-curve, learned (variational autoencoder) and human-derived (Stitch), form a controlled realism spectrum. A Random Forest detector is evaluated under a zero-shot protocol with player-disjoint splits and thresholds calibrated solely from human scores in the target game across three transfer directions. Raw features transfer unreliably, with four of twelve combinations remaining usable. The failure modes can be categorised into three types: ranking inversion, threshold collapse and signal loss. In contrast, the ten-dimensional scale-invariant features achieve a 100% calibrated detection rate for every attack generated from scratch across all three transfer directions with false-positive rates of no more than 5.1% and AUC values of at least 0.958. Simple attacks require only four ratio features, whereas the learned attack requires the micro-structure features in the full ten-dimensional set. Consequently, attack realism determines the required feature granularity rather than detectability itself. Cross-genre transfer from FPS to MOBA is no worse than within-genre transfer. However, the replay-style Stitch attack remains undetectable across games, defining a capability boundary and motivating a layered defence. Overall, the results demonstrate that cross-game zero-shot bot detection is feasible for attacks generated from scratch, with threshold calibration requiring only unlabelled human data and thereby providing a deployable solution to the cold-start problem in anti-cheat systems.

Contents

Honour Code	ii
Abstract	iii
List of Tables	vii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement and Research Questions	2
1.3 Thesis Outline	3
2 Background and Literature Review	4
2.1 Mouse Dynamics as a Behavioural Signal	4
2.2 Mouse-Based Bot Detection	5
2.3 Synthetic Trajectory Generation	7
2.4 The Cross-Game Challenge and Research Gap	9
3 Methodology	11
3.1 Datasets	11
3.1.1 Red Eclipse	12
3.1.2 CS:GO	12
3.1.3 LoL	13
3.2 Feature Extraction	13
3.3 Bot Generation	15
3.3.1 Scripted Bot	16

3.3.2	Bézier Bot	16
3.3.3	VAE Bot	17
3.3.4	Stitch Bot	17
4	System Evaluation	18
4.1	Classifier	18
4.2	Evaluation Metrics	19
4.3	In-Domain Evaluation	19
4.4	Cross-Game Evaluation	20
5	Experimental Results	21
5.1	In-Domain Detection	21
5.2	Cross-Game Transfer with Raw Features	22
5.3	Cross-Game Transfer with Scale-Invariant Features	24
5.4	Diagnostic Analysis	25
6	Discussion, Limitations and Future Work	29
6.1	Principal Findings	29
6.2	Limitations	30
6.3	Future Work	31
7	Conclusion	33
	References	34
	Appendices	37
	GitLab Repository	37

List of Tables

3.1	Extracted features.	15
3.2	Scale-invariant features.	15
5.1	In-domain detection per 10-second window. AUC in parentheses.	22
5.2	In-domain detection per session (window scores averaged). AUC in parentheses.	22
5.3	Cross-game transfer with Raw features (6). Values are detection rate / false-positive rate.	23
5.4	Cross-game transfer with scale-invariant minimal features (SI-min, 4). Values are detection rate / false-positive rate.	25
5.5	Cross-game transfer with scale-invariant extended features (SI-ext, 10). Values are detection rate / false-positive rate.	25
5.6	Feature medians of humans and synthetic bots.	27
5.7	Bézier Red Eclipse → LoL inversion diagnostic.	28
5.8	SI-ext feature importance (VAE, Red Eclipse).	28
5.9	VAE ablation on SI-ext: AUC / calibrated detection rate.	28

List of Figures

1	Processing pipeline from raw game data to evaluation.	38
2	Cumulative mouse paths over one three-minute Red Eclipse session for a human player and for each generator. Axes are scaled independently per panel.	39
3	Detector score distributions and threshold sweeps for three cross-game cases. From top: Scripted Red Eclipse to CS:GO with Raw features, Bézier Red Eclipse to LoL with Raw features, and the same Bézier case with SI-min.	40
4	Detector score distributions and threshold sweeps for VAE Red Eclipse to CS:GO with SI-min (top) and SI-ext (bottom).	41

CHAPTER 1

Introduction

1.1 Motivation

Online games have long faced threats from cheats and game bots. Tools such as aimbots and automated farming bots undermine competitive fairness and directly affect player retention and game revenue [1, 2]. Conventional anti-cheat systems primarily scan for signatures of known cheat programs, but cheat developers can evade detection by obfuscating their code. This limitation has driven a shift from code-based scanning towards player-behaviour analysis in both academia and industry. Human hand movements are constrained by neuromuscular physiology, producing characteristic patterns in movement speed, path curvature and correction rhythm that are difficult to reproduce completely (see Section 2.1 for the background). Mouse dynamics therefore supports two major application areas: identity authentication and human-versus-bot detection.

However, behavioural signals are highly sensitive to their environment. Mouse sensitivity, screen resolution, hardware differences, logging mechanisms and task conditions can systematically alter their numerical characteristics [3]. Consequently, a model trained in one environment may fail in another, with the specific failure mode difficult to predict in advance.

In game anti-cheat, this limitation creates a practical problem: detection systems are developed on a game-by-game basis. Because these methods depend on in-game data, each game requires its own dedicated cheat detection system [4]. This creates a cold-start problem. When a new game launches, no game-specific cheat samples are available for training, so the detector must accumulate them from scratch. However, if human-machine behavioural differences are shared

across games, a detector trained on an older game should transfer directly to a new one. This study therefore investigates whether such cross-game transfer is feasible, under what conditions it succeeds and which attacks it can detect. These questions have not yet been systematically addressed.

1.2 Problem Statement and Research Questions

Cross-game transfer remains an open question in mouse-behaviour research. Kanervisto et al. [2], who investigated mouse-based bot detection in games, note that their experiments were conducted in a single game environment. They suggest that the results may extend to other games within the same genre but may not generalise across genres or input methods. Concurrently, the study introducing the Red Eclipse dataset used here also identifies generalisation to other games as an area for future work [5]. To our knowledge, no study has evaluated whether a detector trained on game A to distinguish human from synthetic trajectories can be deployed directly on game B without retraining.

This question is challenging for several reasons. Mouse signals differ across games in units (view angles versus screen pixels), logging mechanisms (event-driven versus fixed rate), player populations and sensitivity settings, producing substantial shifts in feature distributions. Real cheat data are also difficult to obtain, and their realism cannot be systematically controlled. We therefore adopt synthetic mouse trajectories as attack proxies, covering four levels of realism. This approach also addresses the limited systematic evaluation of behavioural detection systems against synthetic attacks, identified by Khan et al. [3] as a distinct evaluation dimension via spoof attacks.

Using three public game datasets (two first-person shooters (FPS) and one multiplayer online battle arena (MOBA)) and four synthetic attacks with increasing realism, this study conducts a systematic empirical evaluation guided by two research questions.

RQ1. Can a bot detector trained on one game detect synthetic bots in a second, unseen game without retraining? If so, which feature representation enables transfer: raw or scale-invariant features? We evaluate both within-genre transfer (FPS to FPS, in both directions) and cross-genre transfer (FPS to MOBA).

RQ2. How does the trajectory-generator realism affect detection difficulty and cross-game transferability? We construct a controlled realism spectrum comprising four generator types (rule-based, parametric-curve, learned and human-derived) and examine each type separately.

1.3 Thesis Outline

The rest of this thesis is organised as follows. Chapter 2 reviews mouse dynamics as a behavioural signal, human-versus-bot detection and synthetic trajectory generation, and identifies the cross-game research gap. Chapter 3 describes the three datasets and the pipeline that unifies them, three feature sets (the raw set Raw, basic scale-invariant set SI-min and extended scale-invariant set SI-ext) and four methods employed to generate synthetic bots. Chapter 4 defines the evaluation protocol, including the classifier, leakage-safe in-domain cross-validation, zero-shot cross-game setting and AUC and calibrated-threshold metrics. Chapter 5 presents the experimental results, including the in-domain baseline, cross-game results with raw features and their failure patterns, main results with scale-invariant features and feature-level diagnostic analysis. Chapter 6 discusses the implications of the findings, limitations and future directions. Chapter 7 concludes the thesis.

Background and Literature Review

2.1 Mouse Dynamics as a Behavioural Signal

Mouse dynamics describes the behavioural characteristics of how a person operates a mouse, including movement speed, path curvature and pause rhythm. Early studies by Pusara and Brodley [6] and Gamboa and Fred [7] indicated that mouse trajectories contain distinguishable individual behavioural patterns. Ahmed and Traore [8] subsequently established mouse dynamics as a non-intrusive technique for user identification. Unlike physiological biometrics, such as fingerprints and irises, behavioural biometrics capture how a person acts. Their key practical advantage is passive monitoring, which allows continuous observation without interrupting the user [8]. This capability is not available with one-time authentication mechanisms, such as passwords or fingerprints.

The identifiable signal information in mouse trajectories arises from regularities in human motor control. Fitts' law states that the time required for a pointing movement is systematically related to the distance and size of the target [9]. Flash and Hogan [10] revealed that human arm movements follow the minimum-jerk principle, producing smooth, approximately bell-shaped velocity profiles characterised by an initial acceleration followed by deceleration. Human hand trajectories are therefore not arbitrary but constrained by the physical and physiological properties of the neuromuscular system. This principle extends to mouse dynamics. Khan et al. [3] review the cognitive and motor foundations of mouse behaviour, including Fitts' law, while Acien et al. [11] derive mouse-dynamics features directly from a neuromotor model. Zheng et al. [12]

further demonstrated that trajectory measures, especially angle-based features, can distinguish individuals sufficiently well to support identity verification.

Early applications of mouse dynamics focused on identity authentication, including intrusion detection [8], re-authentication [12] and continuous authentication. Continuous authentication verifies the operator’s identity via mouse behaviour throughout a session after login, helping to protect against session hijacking [12, 13, 14]. Khan et al. [3] provide a broader survey of this research. More recently, this authentication paradigm has been extended to game settings. Siddiqui et al. [15] trained a binary Random Forest classifier for each player in Minecraft for continuous authentication, demonstrating that mouse dynamics can provide a useful identity signal within a game environment.

Beyond identity authentication, mouse behaviour can also distinguish human operators from automated programs. Applications include web bots, CAPTCHA [16, 17, 11], game anti-cheat and bot detection [1, 18, 2]. In games, conventional anti-cheat systems primarily rely on signatures, scanning memory for known cheat-program signatures. This approach can be readily circumvented via code obfuscation [2]. Behaviour-based detection therefore provides an alternative by assessing whether player behaviour exhibits human characteristics. A representative early approach is the Human Observational Proofs (HOPs) proposed by Gianvecchio et al. [1], which passively monitor input behaviour and exploit the difficulty bots face in reproducing human-level input. An industry example is Valve’s VACNet, which analyses aiming behaviour using a deep learning model [2].

2.2 Mouse-Based Bot Detection

Existing studies that separate humans from programs by mouse behaviour can be grouped into two settings: web and games. In the web setting, Chu et al. [16] developed a decision-tree detector that combines mouse and keyboard events to identify blog bots and defined three attack types: Form Injection, Human Mimic and Replay Bot. Iliou et al. [17] combined web logs with mouse behaviour to detect advanced web bots. Acien et al. [11] focused on detecting synthetic mouse trajectories using BeCAPTCHA-Mouse. In the game setting, Gianvecchio et al. [1] were among the earliest to apply the HOP concept to World of Warcraft, detecting bots from mouse and keyboard actions. Kanervisto et al. [2] focused on FPS aimbots and are therefore particularly relevant to the present study in terms of game genre. Zhao et al. [18] developed the T-Detector using real massively multiplayer online role-playing game (MMORPG) data from NetEase, combining mouse trajectories with in-game character trajectories. Other studies rely on game logs rather than mouse input. For example, Lee et al. [19] validated a bot-detection framework across three commercial MMORPGs.

The analysis workflow in this field follows a largely consistent pipeline. Raw data typically comprise time-stamped cursor coordinates (t, x, y) , sometimes accompanied by action types [12]. First, the mouse trajectory is cleaned by removing sudden jumps and hardware-induced outliers [8]. The continuous trajectory is then divided into segments that serve as the sample units. Segmentation generally follows three approaches: combining one movement with one click, using fixed-size segments, or adopting trajectories that are already represented as discrete point-to-point sequences, as in CAPTCHA settings. Fixed-size segmentation has no standard length and represents a trade-off between observation duration and detection accuracy. Antal and Fejér [20] adopted blocks of 128 mouse events, Gianvecchio et al. [1] used a fixed number of actions, Kanervisto et al. [2] employed fixed-length event sequences and Antal et al. [21] represented each mouse action using 128 points. Zhao et al. [18] argued that the 10–15 min observation periods adopted in earlier studies are impractical for real-time detection and instead used sequences averaging approximately 90 s. Conversely, Antal and Egyed-Zsigmond [22] demonstrated that longer observations improve detection accuracy, confirming that segment length involves a trade-off. The next stage follows either a feature-based or sequence-based approach. Feature-based methods summarise each segment with statistical descriptors and remain the most common, and sequence-based methods feed the raw (dx, dy) sequence directly into a deep learning model. Both approaches are compared by Iliou et al. [17] and Antal et al. [21]. The final stage makes the detection decision with one of three common strategies. Binary classification trains a model using human and bot samples, with decision trees, Random Forests and support vector machines (SVMs) among the commonly adopted classifiers [16, 17, 11]. One-class or anomaly detection instead models normal human behaviour and is suitable when representative samples of all bot variants are unavailable [13], and the evaluation in Antal et al. [21] also uses this setting. Similarity matching directly compares trajectories, typically using dynamic time warping (DTW); for example, ReMouse adopts trajectory similarity to detect replay bots that closely reproduce human movements [23].

This study adopts the feature-based approach for the third step. Existing features can be broadly grouped into four classes. The first comprises kinematic statistics, including the mean, standard deviation and quantiles of speed, acceleration, step length and direction. This is the earliest and most general feature class [7, 8, 22]. The second includes angle-based features, which capture direction and curvature. Zheng et al. [12] demonstrated that these measures are largely unaffected by the computing platform and remain stable across platforms. Unlike speed, which can be affected by screen resolution and mouse settings, direction and curvature angles are less sensitive to such factors, making them particularly relevant to cross-environment applications. The third class comprises efficiency and regularity features, which are designed to capture bot-like movement patterns. Gianvecchio et al. [1] defined movement efficiency as straight-line displacement divided by the actual path length and found that bots produced substantially straighter movements than humans. Specifically, 81.7% of bot mouse movements had perfect

efficiency, compared with 14.1% of human movements. Chu et al. [16] reported similar patterns for scripted bots, which more frequently follow straight paths, maintain constant speeds and generate evenly spaced events. The fourth includes procedural features, which describe the overall structure of a movement. Shen et al. [24] argued that retaining the shape of the complete velocity curve can be more informative than relying on single-point statistical summaries. An alternative to feature-based detection is the sequence-based deep learning approach, which provides raw sequences or trajectory images to models such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs). For example, Chong et al. [25] adopted this approach, whereas the mouse module of Iliou et al. [17] processes trajectory images with a CNN. Zhao et al. [18] similarly employed a pre-trained model.

At the final decision stage, tree-based models are widely used. Acien et al. [11] compared multiple classifiers and selected Random Forest because it achieved the best performance. Kanervisto et al. [2] similarly reported that Random Forest, SVM and deep neural networks performed comparably for game-bot detection. Random Forest is also adopted as a baseline in several studies [14, 22, 17]. Beyond binary classification, two alternative approaches are one-class anomaly detection and similarity matching. One-class methods model only human behaviour and treat deviations from this distribution as suspicious. Similarity-based methods directly compare trajectories.

The literature commonly evaluates detection performance with area under the receiver operating characteristic curve (AUC) or receiver operating characteristic (ROC) curves [22, 21, 18, 14, 19]. Conventional biometric measures, including equal error rate (EER), false acceptance rate (FAR) and false rejection rate (FRR), are also reported [14, 24], while the area under the precision–recall (PR) curve (PR-AUC) is adopted under class imbalance [18]. Validation also requires careful control of data leakage. Feature engineering and time-series cross-validation can allow micro-structural features from the same player to appear in both the training and test sets, inflating performance. Rigorous studies therefore adopt subject-disjoint protocols. Iliou et al. [17] employed a strictly user-independent evaluation, while Pinto et al. [4] systematically defined and implemented player-based cross-validation. Kanervisto et al. [2] likewise enforced player-wise splitting between training and test sets to ensure that the classifier learned human–bot behavioural differences rather than player-specific operating habits.

2.3 Synthetic Trajectory Generation

Bot detection research faces a structural challenge: real-world malicious bot trajectories are extremely difficult to obtain and label. Consequently, most studies synthesise bot samples themselves. Synthetic data serve two purposes [11]; they act as attacks for evaluating detector robustness and as training data for diversifying the negative class. Acien et al. demonstrated that detectors trained only on human data achieved less than 70% accuracy against most attacks,

whereas adding synthetic samples increased accuracy to as high as 98.7% even against highly realistic attacks.

Synthetic mouse-trajectory generation methods can be arranged on a spectrum from mechanical to human-like behaviour. They fall into two broad classes: generated-from-scratch methods, which produce trajectories with predefined rules or generative models, and human-derived methods, which use human trajectories as source material. The following methods are presented according to their prevalence in the literature.

Rule-based or scripted generation is common across detection studies because it provides a simple lower-bound attack and a readily controlled baseline. Typical implementations adopt straight-line movements with fixed or random step lengths. Iliou et al. [17], for example, employed a moderate bot that moves towards a target in straight lines with step length 1, while their advanced bot introduces a reading-like rhythm and random offsets. Niu et al. [26] simulated human behaviour with four bot types, including straight-line and semi-straight movements. Chu et al. [16] developed a Human Mimic Bot that generates mouse events via the operating system API using AutoHotkey.

Bézier curves are a standard parametric baseline for human-like trajectory generation. They are commonly treated as a lower bound of reasonable human likeness, against which more advanced generation methods are evaluated. Antal et al. [21] used quadratic (random control points) and cubic (the HumanCurve class) Bézier curves from the pyclick library as baselines for SapiAgent. Acien et al. [11] proposed another function-based approach that independently combines three trajectory shapes (linear, quadratic, exponential) with three velocity profiles (constant, logarithmic, Gaussian), producing a 3×3 grid of combinations. They found that the velocity profile has the strongest effect on detectability, with profiles incorporating initial acceleration and final deceleration being the most difficult to detect.

Among learned generative methods, generative adversarial networks (GANs) are the most common. These models learn the distribution of human trajectories and generate novel samples from the learned distribution. Acien et al. [11] employed a long short-term memory (LSTM) GAN that learned the initial acceleration and the fine end-of-trajectory corrections. Kanervisto et al. [2] developed a Wasserstein GAN for an FPS setting that generates aiming corrections over human gameplay. Another learned approach is SapiAgent of Antal et al. [21], which employs an autoencoder to transform straight trajectories into human-like movements that are difficult for anomaly detectors to distinguish.

Human-derived generation represents the strongest attack in the threat model. Rather than generating trajectories from scratch, these methods use human data directly. Chu et al. [16] developed a Replay Bot that records and replays human actions and found it to be the hardest of their three bot types to detect because every replayed action originates from genuine human input. ReMouse [23] similarly investigates session-replay bots and argues that even repeated

human actions are unlikely to produce nearly identical trajectories. Thus, excessive trajectory similarity can itself reveal replay attacks, which ReMouse detects with DTW distance.

Among the studies reviewed, no work adopts a variational autoencoder (VAE) to generate mouse trajectories. Although VAEs are widely used for other generative tasks, no direct precedent was identified for mouse-trajectory generation in the literature above. In addition, most studies evaluate only one or two points on the realism spectrum, typically their proposed method and a single baseline. Few studies compare rule-based, parametric-curve, learned and human-derived generators under a common detection protocol.

2.4 The Cross-Game Challenge and Research Gap

Cross-game deployment introduces several sources of domain shift. Initially, coordinate units differ: desktop environments record pixel coordinates, whereas FPS games typically express mouse input as view angles (yaw and pitch). The three datasets used here happen to span both representations [27, 5]. Next, logging mechanisms differ. Event-driven systems record data only when actions occur; for example, mouse and key presses in AMuCS are logged at irregular intervals, producing a time-series structure that differs substantially from fixed-rate sampling. Subsequently, user-side settings vary. Screen resolution, mouse sensitivity and hardware can systematically alter speed-related features. Zheng et al. [12] therefore argued for angle-based features, while Shen et al. [24] systematically identified these confounding factors and demonstrated the importance of controlled environments. Furthermore, player populations and task structures differ across games, leading to distinct operating behaviours [28].

Existing cross-environment research remains limited. Related fields have made several attempts, but none directly addresses cross-game transfer for pure mouse dynamics. Wang et al. [29] proposed scene-irrelated features and user-related features (SIURUA) for authentication by adopting both keyboard and mouse data. Although these features are conceptually similar to the scale-invariant features used here, their goal is transfer across task scenarios rather than games. Recent work on cross-environment mouse and behavioural trajectories has also explored self-supervised learning and pre-training. Mouse2Vec [30] achieved mouse representation transfer across datasets, while T-Detector [18] demonstrated method-level generalisation to data from a second MMORPG. However, Mouse2Vec targets user identification rather than human-versus-bot detection and relies on a complex self-supervised model. T-Detector, meanwhile, uses multimodal data, such as in-game character movement coordinates, and requires substantial pre-training in a specific game environment. Thus, direct cross-game human-versus-bot detection from pure mouse trajectories, without extensive pre-training or fine-tuning, remains largely unexplored.

The only prior work identified that directly examines cross-game transfer is a side experiment by Gianvecchio et al. [1]. They trained a model on World of Warcraft and tested it on Diablo 2 bots

without retraining, demonstrating that the model captured behavioural invariances shared across games. However, this was a small supplementary experiment and used a real scripted bot rather than modern synthetic trajectories. Zhao et al. [18] likewise noted that existing handcrafted features are often attached to specific games and datasets, limiting their generalisability. Hence, whether handcrafted features can transfer across games requires empirical validation rather than assumption.

These issues have direct practical relevance because each game typically requires its own detection system [4]. Although Pinto et al. claimed their method could be applied across games and input methods, their experiments were conducted within a single game. Cross-game transfer has therefore remained largely at the level of claims or game-specific redevelopment and has not been systematically evaluated under a zero-shot setting.

Three gaps emerge from Section 2.2 - 2.4. First, although mouse bot detection is well established within individual environments, zero-shot transfer between games has not been systematically evaluated. Subsequently, although synthetic attack methods span different levels of realism, few studies have compared a controlled four-family spectrum under the same protocol to assess realism, detectability and transferability. Furthermore, no study has combined scale-invariant features with threshold calibration requiring no bot labels to provide a complete evaluation of cross-game deployment. The research questions posed in Chapter 1 correspond to these gaps.

CHAPTER 3

Methodology

This chapter describes the materials and procedures of the study. The complete processing pipeline, from raw game data to evaluation, is summarised in Figure 1.

3.1 Datasets

This study adopts two game genres, FPS and MOBA, represented by three public datasets: the FPS games Red Eclipse and Counter-Strike: Global Offensive (CS:GO), and the multiplayer online battle arena League of Legends (LoL).

None of the datasets was originally collected to distinguish human from machine-generated mouse trajectories. Instead, they were developed for rapid skill capture in an FPS [5], affective computing [27] and stress response modelling [31]. Nevertheless, all three datasets comprise continuous mouse trajectories or in-game view data recorded during real gameplay, providing the behavioural signal required here.

The datasets differ in gameplay, rules and player operating behaviour. They also employ different recording formats (mouse displacement, screen cursor coordinates and in-game view vectors) and different sampling rates and units (pixels and degrees). These differences require data harmonisation before cross-game analysis. Accordingly, all datasets are converted into a common representation: a displacement sequence with a time field, (dx, dy, t) .

The following sections describe each dataset and its conversion to this unified format.

3.1.1 Red Eclipse

The Red Eclipse dataset [5] comprises 476 three-minute games from 45 subjects. The game engine records raw mouse movement as integer pixel displacements, with each mouse movement event containing the horizontal and vertical displacements, x and y , in pixels. Logging is event-driven and records data only when the mouse moves. Because each game is short and involves continuous combat, this study treats each complete game as one trajectory without further segmentation. As the simplest of the three datasets, Red Eclipse provides the reference format for the data fields and trajectory length adopted to harmonise the other two datasets.

3.1.2 CS:GO

For CS:GO, this study employs the AMuCS dataset [27], which comprises multiple signals from 256 subjects across 71 experimental sessions, with each session involving two or four players. Although the dataset includes a mouse coordinate file (`mouseposition.csv`), the official README states that the game engine intercepts and resets the cursor during play, making these coordinates unsuitable for reconstructing mouse movement. This study therefore uses the three-dimensional in-game view direction vector (`eyeVectorX/Y/Z`) recorded in `gameFlt.csv` as a proxy for the mouse movement. Frame-to-frame changes in yaw and pitch, measured in degrees, provide the (dx, dy) fields of the unified representation. In a first-person shooter the view direction is controlled by the mouse, so this vector can serve as an equivalent mouse signal.

The conversion from unit vectors to view angles follows standard computer-graphics conventions¹. Pitch is computed from the arcsine of the vertical component, while yaw is obtained with the two-argument arctangent of the horizontal components. Differences between consecutive angles then provide the equivalent mouse movement. Yaw wrapping at $\pm 180^\circ$ is corrected to prevent boundary crossings from being interpreted as large mouse movements. Frame-to-frame changes exceeding 30° , which correspond to player respawns or switches to spectator view rather than genuine aiming movements, are removed; these account for approximately 0.6% of the data. The conversion is validated in two ways: converting synthetic vectors to angles and back recovers the original values almost exactly, while reconstructing real view vectors from the converted data produces errors below 10^{-5} .

After conversion, the warm-up round is excluded, and trajectories are extracted from the second round onwards. Player health recorded in `gameInt.csv` is used to retain only periods in which the player is alive ($\text{health} \geq 1$). Only 3-min segments are retained, and sessions containing fewer than two events are removed. This procedure yields 237 mouse trajectories.

¹The conversion formulas follow the Camera chapter of LearnOpenGL (<https://learnopengl.com/Getting-started/Camera>).

3.1.3 LoL

The LoL dataset [31] was originally collected to study stress responses during competitive play and comprises keyboard, mouse and physiological data from eight players. Mouse activity is recorded as screen-cursor coordinates whenever the mouse moves.

Unlike the two datasets described above, the LoL dataset records continuous play periods lasting several hours, including multiple matches, waiting and resting periods in the lobby. The actual matches must therefore be identified separately. The timestamps and match durations in the accompanying `matchid-*.json` log files are used to locate the gameplay periods, after which the dataset’s prescribed time correction is applied.

Because gameplay activity is relatively slow during the opening stages of a MOBA match, this study extracts a fixed three-minute segment from minutes 10 to 13 of each match. Matches shorter than 13 minutes or segments containing fewer than 100 mouse-movement events are discarded. This yields 59 trajectories from seven players across 16 matches. The cursor coordinates are then converted into screen-pixel displacements by differencing consecutive observations.

3.2 Feature Extraction

After the processing described in Section 3.1, trajectories from all three datasets are represented as (dx, dy, t) displacement sequences. Each trajectory is then converted into a fixed-length feature vector for classification. Three feature sets are constructed. The first comprises 14 features for in-domain detection, with six selected to form the raw cross-game set (Raw). The second uses feature ratios to reduce differences in absolute scale between games. This scale-invariant representation is defined in two forms: a core four-dimensional set and an extended ten-dimensional set.

The raw input comprises time-stamped mouse events. Subtracting adjacent events yields the displacement sequence $(\Delta t, dx, dy)$, from which four intermediate sequences are derived: step distance, speed, acceleration and turning angle. Step distance is $\sqrt{dx^2 + dy^2}$, while speed is calculated as distance divided by time gap (Δt) , using only observations with $\Delta t > 0$. Movement direction is obtained from $\text{atan2}(dy, dx)$, and the angle between two adjacent step directions gives the turning angle, with wrap-around at $\pm 180^\circ$ corrected. Events with $\Delta t = 0$ are excluded when computing speed and acceleration. Statistical measures, including the mean, standard deviation and maximum, are then applied to these sequences to obtain the final feature values.

The in-domain feature set (14 dimensions) is used for detection within individual games. Because all sessions within a game share the same units and logging mechanism, absolute features can be adopted directly. The set combines mouse-related features available in the Red Eclipse dataset [5] with commonly employed mouse trajectory features from the literature [7, 3, 18, 12, 1, 2]. The 14 features comprise three groups (Table 3.1). The first contains basic kinematic statistics: average

speed, idle ratio (`idle_ratio`), average turning angle, speed standard deviation, maximum speed and step-distance standard deviation. The next includes protocol quantities tied to session length and sampling settings: total movement, number of events and average time gap. The final comprises unit-free structural and ratio features: subsegment movement efficiency (`efficiency_sub`), turning angle standard deviation, vertical–horizontal ratio (`vh_ratio`), direction change rate (`dir_change_rate`) and two-axis correlation (`xy_corr`). Movement efficiency is conventionally defined for a single point-to-point action [1]. Applying it directly to a continuous three-minute trajectory would cause most values to approach zero, reducing its discriminative value. Consequently, this study divides each trajectory into subsegments of 40 events, computes the ratio of start-to-end straight-line distance to actual path length for each subsegment, and averages the resulting values.

The Raw set comprises the six basic kinematic statistics selected from the fourteen-dimensional in-domain set, with no scale correction. Because the units (pixels versus degrees) and value ranges of the three games can differ by a factor of up to 100, a model trained on these features may learn game-specific characteristics rather than human–bot behavioural differences. This set therefore provides a direct comparison between raw absolute-value features and the subsequent standardised features, motivating the design of the scale-invariant feature sets. Protocol-dependent features are excluded because they are directly influenced by session length and logging mechanisms and would therefore encode each game’s sampling characteristics. Structural and ratio features are reserved for the scale-invariant sets. This design isolates kinematic scale as the principal source of cross-game mismatch.

The core cross-game feature set converts absolute values into ratios, reducing sensitivity to differences in feature magnitude and causing units to cancel during division. This four-dimensional set, termed scale-invariant minimal (SI-min), comprises speed coefficient of variation (`speed_cv`), calculated as speed standard deviation divided by average speed; speed peak ratio (`speed_peak`), computed as maximum speed divided by average speed; average turning angle, expressed in radians and therefore unitless; and the acceleration coefficient of variation (`accel_cv`).

SI-min is compact and less prone to overfitting when player samples are limited, but it is insufficiently stable for detecting VAE-type synthetic data. To test whether the results depend on the specific feature selected for SI-min, an extended scale-invariant set is also constructed. Six additional unitless features are added: turning-angle standard deviation, subsegment efficiency, vertical–horizontal ratio, direction-change rate, two-axis correlation and step-to-step autocorrelation. Together with SI-min, these form the scale-invariant extended (SI-ext), comprising ten dimensions (Table 3.2).

The two sets serve complementary purposes. SI-min provides a compact baseline for determining how far cross-game detection can be achieved with minimal features, whereas SI-ext tests whether additional micro-structural information improves detection of more challenging attacks, especially

VAE-generated trajectories. Comparing both sets prevents improvements from being attributed simply to increasing feature dimensionality.

Table 3.1: Extracted features.

Feature	Code name	Description	Cross-game
Average speed	avg_speed	Mean of step distance / time gap	✓
Idle ratio	idle_ratio	Fraction of steps with distance < 1	✓
Average turning angle	avg_turn_angle	Mean $ \Delta\text{heading} $ between steps	✓
Speed variation	speed_std	Std of per-step speed	✓
Maximum speed	speed_max	Max per-step speed	✓
Movement variation	dist_std	Std of per-step distance	✓
Total movement	total_movement	Sum of step distances $\sqrt{dx^2 + dy^2}$	
Number of events	n_events	Count of mouse events	
Average time gap	mean_dt	Mean interval between events	
Subsegment efficiency	efficiency_sub	Mean straightness over 40-event chunks	
Turning angle std	turn_angle_std	Std of $ \Delta\text{heading} $	
Vertical–horizontal ratio	vh_ratio	$\Sigma dy /\Sigma dx $	
Direction-change rate	dir_change_rate	dx sign-flip rate	
XY correlation	xy_corr	Pearson $\text{corr}(dx, dy)$	

Table 3.2: Scale-invariant features.

Name	Formula	Meaning	SI-min	SI-ext
speed_cv	speed_std / avg_speed	Speed consistency	✓	✓
speed_peak	speed_max / avg_speed	Peak speed relative to average	✓	✓
turn_angle_mean	avg_turn_angle (unchanged)	Mean heading change (rad)	✓	✓
accel_cv	$\text{std}(\Delta\text{speed}/\Delta t) / \text{mean}(\Delta\text{speed}/\Delta t)$	Acceleration irregularity	✓	✓
turn_angle_std	std(turn angles)	Variability of turning		✓
efficiency_sub	mean path-efficiency over 40-event chunks	Local straightness		✓
vh_ratio	$\Sigma dy /\Sigma dx $	Vertical vs horizontal motion		✓
dir_change_rate	(# of dx sign flips) / n_events	Horizontal direction reversals		✓
xy_corr	$\text{corr}(dx, dy)$	Axis coupling of steps		✓
step_autocorr	$0.5 [\text{corr}(dx_t, dx_{t+1}) + \text{corr}(dy_t, dy_{t+1})]$	Step-to-step persistence		✓

3.3 Bot Generation

As described in Chapter 2, real cheat data are difficult to obtain, typically unlabelled and cannot provide a controlled comparison across all three games. This study therefore generates synthetic mouse trajectories to simulate game-cheat attacks, following common practice in the literature [11, 21, 2]. Because real-world malicious attacks are hard to collect and label, most mouse-bot detection studies generate their own bot data [17, 16].

Four commonly implemented classes of synthetic trajectories are considered: rule-based, parametric-curve, learned and human-derived. Together, they form a controlled realism spectrum from mechanical to human-like behaviour and represent four corresponding threat types: scripted cheats, curve-based evasion tools, AI-generated imitation attacks and replay-style attacks. Figure 2 shows a human and the trajectories of each generator in the Red Eclipse.

All four methods follow the same calibration principle: parameters are estimated from the human data of the same game rather than from fixed cross-game constants. Time gaps (dt) are sampled from the empirical distribution of human sessions, allowing each synthetic trajectory to reproduce the temporal rhythm of an individual player rather than a mixture of players. Step distances and movement directions are sampled from pooled human observations, with step distances restricted to the 25th–90th percentiles to prevent rare extremes from being reproduced. Synthetic trajectory lengths are aligned to the median length of human sessions in each game. All outputs use the unified (dx, dy, t) format and pass through the same feature-extraction pipeline as the human trajectories.

3.3.1 Scripted Bot

For the rule-based attack, each segment samples a heading and step length and holds both constant for 20–60 events. Gaussian jitter is then added to every step, with a magnitude equal to 0.1 times the median human step length for the corresponding game. A new heading and step length are sampled after each segment, and the process continues until the target trajectory length is reached. This design follows the common rule-based control group used in the literature (the moderate bot of Iliou et al. [17] and the straight-line and semi-straight types of Niu et al. [26]). Its primary detectable characteristic is excessive regularity and straightness. Chu et al. [16] similarly found that scripted bots tend to move along straight paths at constant speeds with evenly spaced events, producing regularities rarely observed in human movement.

3.3.2 Bézier Bot

The parametric-curve attack connects successive strokes using cubic Bézier curves. Each stroke is directed towards a randomly selected target, with sampling points distributed via an ease-out schedule to reproduce the rapid initial movement and gradual deceleration characteristic of human aiming. Perpendicular tremor is added with a magnitude of 0.1 times the median human step length, while curvature is controlled by $\text{bend_scale} = 0.15$. For event-driven games, points that round to $(0, 0)$ are discarded, reflecting the fact that sub-pixel movements are not recorded by event-driven loggers. Here, the Bézier bot and the Scripted bot share the same time, step distance, direction pools, stroke-length range and tremor scale. Their only differences are path geometry and speed structure, enabling a controlled comparison. Bézier curves provide a recognised humanisation baseline [21] and are also adopted in practical evasion tools, including

pyclick’s HumanCurve and Ghost cursor. Their main detectable weakness is excessively smooth curvature: human trajectories contain neuromotor variability, whereas an ideal Bézier arc is geometrically regular.

3.3.3 VAE Bot

A VAE is trained separately for each game as a multilayer perceptron VAE (MLP-VAE; latent dimension $z = 12$, β -VAE weighting with $\beta = 0.5$ and per-axis normalisation), generating 64-event segments that are then randomly concatenated into a complete session. This assembly mechanism is identical to that used for the Stitch bot, creating a controlled comparison in which the only difference is the source of the segments. The trained model weights are saved as versioned files, and all evaluations load these fixed weights without retraining to ensure reproducibility. VAEs tend to generate samples towards the centre of the learned distribution and may underrepresent the extreme variations present in human trajectories. The resulting giveaway is therefore over-smoothing and distributional averaging. Learned generation in the literature is predominantly GAN-based [11, 2]. The closest precedent to the present approach is the autoencoder of Antal et al. [21], which transforms straight trajectories into human-like ones. However, that method is deterministic and cannot generate novel samples from a latent space. This study extends that approach by adopting a VAE to sample new trajectories directly from a learned latent distribution.

3.3.4 Stitch Bot

Human trajectories are divided into 1.5–2.5 s chunks and randomly reassembled into new sessions. The gap between consecutive chunks is sampled from the human dt distribution of the same game (inter-chunk gap = one human dt draw), ensuring that all inter-chunk intervals are consistent with observed human timing. This procedure is a form of block bootstrap and serves as a simulated replay-style attack. Because every chunk is taken directly from human action, the only detectable artefact is the discontinuity at the chunk boundaries. The method resembles the Replay Bot of Chu et al. [16], which records and replays complete human segments, and the session-replay bots investigated by ReMouse [23]. The key difference from whole-segment replay is that ReMouse-style detection relies on high similarity between a replayed trajectory and an existing record, allowing verbatim replays to be identified. In contrast, randomly reassembled human-derived chunks produce a different trajectory sequence for each session, thereby avoiding direct duplication and evading similarity-based detection.

CHAPTER 4

System Evaluation

This chapter describes how the detection models are trained and evaluated. Two evaluation levels are adopted. In-domain evaluation trains and tests within the same game to verify that the detection method works, whereas cross-game evaluation trains on one game and tests directly on another to address the central research question. Both settings share the same classifier and evaluation metrics, which are introduced initially.

4.1 Classifier

All experiments adopt Random Forest with 100 trees (`n_estimators = 100`). A separate binary classifier is trained for each game and bot type (that game’s humans versus that bot). Random Forest is selected for three reasons. First, it is a standard baseline in mouse-dynamics research [14, 22, 17] and was also used by Buckley et al. [5] on the same Red Eclipse dataset, where its generalisation ability was noted for feature sets with numerous features of uncertain nature. Next, Random Forest is insensitive to the numerical scaling of features, making it suitable for the mixed units encountered in cross-game evaluation, while remaining robust when the feature space and sample size are limited. Finally, it provides feature-importance measures that support interpretation of the behavioural cues employed by the detector. Hence, each game–bot combination has an independent detector trained to distinguish that game’s humans from the corresponding bot type.

4.2 Evaluation Metrics

The main metric is AUC, which measures whether bots generally receive higher suspicion scores than humans across all possible decision thresholds. This makes it suitable for cross-game evaluation, where score distributions may shift between games. In addition to AUC, this study reports the detection and false-positive rates under a calibrated threshold. The threshold is set to the 95th percentile of human scores in the target game; hence, scores above it are classified as bots, and the expected false-positive rate is approximately 5%. Because the threshold is derived solely from human data, no bot labels are required. This reflects the practical cold-start scenario in which a newly launched game provides human data and no labelled cheat samples. A fixed threshold of 0.5 is also evaluated for comparison. Such a threshold can fail under cross-game score shifts: although AUC may remain high, shifted scores can cause most or all samples to receive the same predicted class, making the apparent detection performance collapse. Detailed analyses are presented in Chapter 5.

4.3 In-Domain Evaluation

In-domain evaluation trains and tests the detector within the same game. It verifies whether the detector can distinguish humans from synthetic bots under matched conditions and provides a baseline for assessing cross-game transfer. This evaluation uses the fourteen-dimensional in-domain feature set described in Section 3.2.

In-domain evaluation divides each approximately 3-minute trajectory into non-overlapping 10-second windows, retaining only windows containing at least 30 events. Each window is treated as an independent detection sample, with features extracted once per window. This increases the number of training and evaluation samples; treating each complete trajectory as one sample would leave only 59 LoL trajectories, which is insufficient for stable model development. The window-based approach also better reflects practical anti-cheat deployment, where detection should occur within a short time rather than after a player has completed three minutes of gameplay. In addition to window-level results, session-level performance is reported by averaging the scores of all windows from the same trajectory to obtain a single session score (session mean-of-windows).

Because the Stitch and VAE bots are derived from human data, generating all synthetic bots before splitting could lead to leakage. Test-set bots might contain segments from training-set players, allowing the model to encounter previously observed behavioural patterns and artificially inflating performance. To prevent this, this study first performs user-independent k -fold cross-validation (implemented with scikit-learn’s `GroupKFold`), with player identity defining the groups (subject-disjoint). All data from a given player therefore remain exclusively in either the training or test set. Synthetic bots are then generated separately within each fold: training bots are generated

only from training-group humans, while test bots use only test-group humans. Red Eclipse is split into five folds across its 45 players, CS:GO into four folds following the dataset’s four seat folders and LoL into five folds over its seven players. Because some LoL test folds comprise only one or two players, greater variation among folds is expected; this variation is reported alongside the results in Chapter 5. Each fold produces both window-level and session-level results, which are summarised using the mean and standard deviation across folds.

This protocol follows established measures for preventing leakage. Iliou et al. [17] used user-independent evaluation, ensuring that all sessions from an individual remained exclusively in either training or testing sets. Kanervisto et al. [2] likewise performed player-wise splitting, with four of the eight players used for training and four for testing. They also completed the split before training their two GAN-Aimbots and collected data from entirely new participants in the second collection, preventing the player’s behaviour from appearing on both sides of the evaluation.

4.4 Cross-Game Evaluation

Cross-game evaluation is the core of this study. After training on a source game, each detector is applied directly to a target game without retraining. The source and target games comprise completely disjoint player populations, eliminating player overlap between training and testing. Source-game synthetic bots are generated from all available human data, so no within-fold splitting is required. The source humans and synthetic bots are then combined and split into stratified 80/20 training and source-domain evaluation sets. The model trains on the 80% share, and the remaining 20% is kept as a detection check inside the source domain. The target-game data are used exclusively for testing. Cross-game evaluation adopts a complete trajectory of approximately three minutes as one sample, corresponding to the judgement of a single session at deployment. Three transfer directions are assessed: Red Eclipse to CS:GO and CS:GO to Red Eclipse (within-genre FPS transfer in both directions) and Red Eclipse to LoL (cross-genre, FPS to MOBA). Each direction is evaluated against all four bot types using the six-dimensional Raw, four-dimensional SI-min and ten-dimensional SI-ext feature sets described in Section 3.2.

CHAPTER 5

Experimental Results

Tables 5.1 and 5.2 report the in-domain results for the three games: player-disjoint GroupKFold (five folds for Red Eclipse and LoL, four folds for CS:GO), 10-second windows as the sample unit, AUC and the calibrated detection rate (cal detect, with the threshold pinned so that approximately 5% of humans are false positives).

Tables 5.3 to 5.5 present the three transfer directions in turn (Red Eclipse $\rightarrow$ CS:GO, CS:GO $\rightarrow$ Red Eclipse, Red Eclipse $\rightarrow$ LoL), each including four bots and three feature sets (Raw, SI-min, SI-ext). For all transfer experiments, thresholds are calibrated exclusively from human scores in the target game, following the protocol in Section 4.2.

5.1 In-Domain Detection

The three bots generated from scratch (Scripted, Bézier, VAE) achieve 100% session-level detection in all three games with AUC of 1.000. At the window level, all nine combinations attain AUC values of at least 0.984, with the lowest calibrated detection rate at 87.0%. Red Eclipse and CS:GO remain almost entirely at AUC 1.000 and 100% detection. LoL exhibits greater fold-to-fold variation, particularly for Scripted ($87.0\% \pm 26\%$) and VAE ($90.2\% \pm 14\%$), because it has only seven players. Some folds therefore comprise only one or two players in the test group, making per-fold threshold calibration unstable. Nevertheless, window-level AUC remains at least 0.984, and session aggregation restores 100% detection for all three bots. These results establish a reliable in-domain baseline, confirming that subsequent cross-game failures cannot be attributed to an inability of the classifier to detect the attacks within individual games.

In contrast, the human-derived Stitch bot is substantially harder to detect. Window-level AUC is 0.746 for Red Eclipse, 0.913 for CS:GO, and 0.634 for LoL, with calibrated detection rates ranging from 19.3% to 61.6%. This follows the design of the synthetic trajectories described in Section 3.3. Bots generated from scratch produce machine-like movement throughout the trajectory, so each 10-second window contains detectable evidence. Stitch, however, comprises genuine human segments, with artificiality introduced only at the seams between segments every 1.5–2.5 s. A 10-second window therefore contains relatively little evidence of the attack. Averaging the window scores within each session substantially improves detection: the detection rate increases from 19.3% to 77.2% for Red Eclipse (AUC 0.959), reaches 100% for CS:GO (AUC 1.000) and increases from 19.8% to 32.0% for LoL (AUC 0.751). These results indicate that session-level observation is more appropriate for detecting human-derived attacks such as Stitch, whereas a 10-second window is sufficient for mechanical bots. This finding is consistent with the observation that longer observation periods improve detection accuracy [22] and with the practice of aggregating multiple sample scores for session-level detection [2].

Table 5.1: In-domain detection per 10-second window.
AUC in parentheses.

Game \ Bot	Scripted	Bézier	VAE	Stitch
Red Eclipse (5-fold)	100%±0 (1.000)	99.9%±0.1% (1.000)	100%±0 (1.000)	19.3%±2.4% (0.746)
CS:GO (4-fold)	100%±0 (1.000)	100%±0 (1.000)	100%±0 (1.000)	61.6%±4.2% (0.913)
LoL (5-fold)	87.0%±26% (0.988)	99.8%±0.3% (0.999)	90.2%±14% (0.984)	19.8%±31% (0.634)

Table 5.2: In-domain detection per session (window scores averaged).
AUC in parentheses.

Game \ Bot	Scripted	Bézier	VAE	Stitch
Red Eclipse (5-fold)	100%±0 (1.000)	100%±0 (1.000)	100%±0 (1.000)	77.2%±7.9% (0.959)
CS:GO (4-fold)	100%±0 (1.000)	100%±0 (1.000)	100%±0 (1.000)	100%±0 (1.000)
LoL (5-fold)	100%±0 (1.000)	100%±0 (1.000)	100%±0 (1.000)	32.0%±38% (0.751)

5.2 Cross-Game Transfer with Raw Features

Table 5.3 presents that raw-feature transfer is unreliable. Only four of the twelve combinations achieve a calibrated detection rate of at least 80%, while AUC ranges from 0.242 to 1.000. Failures are not universal: the VAE under Red Eclipse → LoL achieves an AUC of 1.000. However, no consistent pattern indicates which transfer combinations will succeed or how they will fail. For example, the Scripted bot under Red Eclipse → CS:GO retains strong ranking performance (AUC 0.947) but has a calibrated detection rate of 0% because of threshold collapse. In contrast, the Bézier bot under Red Eclipse → LoL yields a reversed ranking, with an AUC of only 0.242. These failures fall into three concrete patterns.

Pattern one: ranking inversion. The Bézier bot under Red Eclipse → LoL achieves an AUC of only 0.242 (Figure 3, second row), clearly below 0.5, indicating that the detector systematically ranks humans as more bot-like than bots. The Stitch bot under CS:GO → Red Eclipse (AUC 0.343) and the VAE under Red Eclipse → CS:GO (AUC 0.384) exhibit the same pattern. This failure is more serious than a complete loss of detection ability because the model produces systematically misleading scores, causing decisions based on those scores to favour the wrong class. The feature mechanism underlying this inversion is analysed in Section 5.4.

Pattern two: score drift and calibration collapse. Several transfer combinations retain good or even excellent AUC but fail at the operating point because the score distributions shift between games. The clearest example is the Scripted bot under Red Eclipse → CS:GO (Figure 3, first row): AUC remains high at 0.947, but all bot scores collapse onto a single value of approximately 0.58, which is also reached by the top 5% of human scores. The calibrated threshold therefore coincides with the bot scores and no bot exceeds it. This confirms the limitation discussed in Section 4.2: fixed thresholds are unsuitable for cross-game detection, while the calibrated threshold is robust but cannot eliminate all failures. The reverse pattern occurs for Scripted under CS:GO → Red Eclipse, where AUC is perfect (1.000), yet a fixed 0.5 threshold would classify 100% of humans as bots because all scores lie above 0.5. Conversely, Bézier under Red Eclipse → CS:GO has AUC 0.970, but a fixed 0.5 threshold detects none of the bots because the entire score distribution shifts downward. Calibration mitigates these failures: using the 95th percentile of target-human scores as the threshold yields 100% detections and 0% false positives for the former case, and 80.2% detection and 4.2% false positives for the latter.

Pattern three: signal loss. Under raw features, the VAE has AUC 0.384 (Red Eclipse → CS:GO) and 0.500 (CS:GO → Red Eclipse) in the two FPS directions. The ranking is completely random, and the boundary the model learned means nothing in the feature space of the unseen game.

Under Raw features, AUC, the fixed threshold and the calibrated threshold can each fail independently. This indicates that cross-game failure is not a single problem but a combination of at least three distinct issues. Addressing only one of them, such as threshold calibration or feature transformation, is unlikely to provide a complete solution.

Table 5.3: Cross-game transfer with Raw features (6).
Values are detection rate / false-positive rate.

Bot	Red Eclipse → CS:GO (both FPS)			CS:GO → Red Eclipse (both FPS)			Red Eclipse → LoL (FPS → MOBA)		
	AUC	Calibrated	Fixed 0.5	AUC	Calibrated	Fixed 0.5	AUC	Calibrated	Fixed 0.5
Scripted	0.947	0% / 0%	100% / 12.2%	1.000	100% / 0%	100% / 100%	0.957	100% / 5.1%	100% / 8.5%
Bézier	0.970	80.2% / 4.2%	0% / 0%	0.761	0% / 0%	100% / 100%	0.242	0% / 5.1%	62.7% / 67.8%
VAE	0.384	0% / 0%	0% / 23.2%	0.500	0% / 0%	0% / 0%	1.000	100% / 1.7%	100% / 42.4%
Stitch	0.470	0% / 1.3%	0% / 0%	0.343	0% / 1.9%	0% / 0%	0.786	23.7% / 5.1%	0% / 0%

5.3 Cross-Game Transfer with Scale-Invariant Features

Tables 5.4 and 5.5 present the results of the same three directions under SI-min and SI-ext.

Within-genre FPS transfer is nearly perfect and requires only four features. Under SI-min, the Scripted and Bézier bots achieve AUC values of at least 0.998 in both Red Eclipse $\rightarrow$ CS:GO and CS:GO $\rightarrow$ Red Eclipse. Calibrated detection rates are 100% in all four cases, while all false-positive rates remain below 5%. Thus, four scale-invariant features are sufficient to transfer detection of both mechanical and parametric-curve bots across FPS games. Notably, this transfer requires no retraining or fine-tuning. After training on the source game, the detector is applied directly to the target game, with the threshold calibrated solely from target-game human scores, despite differences in measurement units (pixels versus degrees) and logging mechanisms (event-driven versus fixed-rate).

Transfer of the learned generator is highly sensitive to feature representation. Under SI-min, its cross-FPS transfer fails (Red Eclipse $\rightarrow$ CS:GO 0.414, CS:GO $\rightarrow$ Red Eclipse 0.692; calibration also breaks down, with the calibrated threshold rising to 0.83 and 1.0 in the two directions). With SI-ext, however, detection recovers completely in both directions (Figure 4), achieving an AUC of 1.000, a 100% calibrated detection rate and false-positive rates of no more than 2.5%. Thus, VAE-generated trajectories are not inherently undetectable. Their distinguishing characteristics, such as over-smoothing and distribution averaging, are not captured by coarse speed-ratio features but emerge in finer behavioural structure, including step-to-step autocorrelation and turning-angle distributions. More realistic attacks therefore require richer feature representations for reliable cross-game detection.

The cross-genre (FPS $\rightarrow$ MOBA) results are almost as good. Under SI-min, Red Eclipse $\rightarrow$ LoL achieves AUCs of 1.000 for Scripted, 0.995 for Bézier and 0.976 for VAE, with calibrated detection rates of 100%, 98.3% and 100%, respectively. For the generators built from scratch, moving between FPS and MOBA introduces no clear additional performance loss. Notably, scale-invariant features rescue the Bézier bot from an AUC of 0.242 under Raw features to 0.995 under SI-min (Figure 3, third row), providing direct evidence for the value of scale-invariant representation. The VAE also transfers successfully in this cross-genre setting without the extended feature set. Its contrast with the failures observed in both within-genre FPS directions indicates that genre similarity does not determine transferability. Instead, transfer depends on whether the attack’s behavioural signature is specific to its source domain.

The Stitch bot remains difficult to detect across all transfer directions and feature sets. Among the six cross-game combinations using scale-invariant features, the highest AUC is only 0.715, while the highest calibrated detection rate is 8.5%. Notably, Raw outperforms SI in one case: Red Eclipse $\rightarrow$ LoL, where AUC reaches 0.786 with Raw features compared with 0.709–0.715 with SI features. Scale invariance therefore cannot resolve the Stitch attack because its distinguishing feature, the seams between human-derived segments, is not caused by differences in feature scale.

Because Stitch consists largely of genuine human movement, a detector that successfully transfers the concept of human behaviour should classify most of it as human. Its only distinguishing signal lies at the segment boundaries, and these seams become less detectable across games as their temporal structure changes.

Table 5.4: Cross-game transfer with scale-invariant minimal features (SI-min, 4).
Values are detection rate / false-positive rate.

Bot	Red Eclipse → CS:GO (both FPS)			CS:GO → Red Eclipse (both FPS)			Red Eclipse → LoL (FPS → MOBA)		
	AUC	Calibrated	Fixed 0.5	AUC	Calibrated	Fixed 0.5	AUC	Calibrated	Fixed 0.5
Scripted	1.000	100% / 4.2%	100% / 0%	1.000	100% / 1.9%	99.2% / 0.4%	1.000	100% / 5.1%	100% / 8.5%
Bézier	1.000	100% / 2.1%	100% / 0%	0.998	100% / 4.0%	98.3% / 3.2%	0.995	98.3% / 5.1%	100% / 40.7%
VAE	0.414	0% / 0%	0% / 13.1%	0.692	0% / 0%	99.2% / 38.4%	0.976	100% / 3.4%	100% / 13.6%
Stitch	0.469	0.8% / 5.1%	0% / 1.3%	0.360	0% / 4.4%	0% / 0%	0.715	8.5% / 1.7%	10.2% / 11.9%

Table 5.5: Cross-game transfer with scale-invariant extended features (SI-ext, 10).
Values are detection rate / false-positive rate.

Bot	Red Eclipse → CS:GO (both FPS)			CS:GO → Red Eclipse (both FPS)			Red Eclipse → LoL (FPS → MOBA)		
	AUC	Calibrated	Fixed 0.5	AUC	Calibrated	Fixed 0.5	AUC	Calibrated	Fixed 0.5
Scripted	1.000	100% / 3.8%	100% / 0%	1.000	100% / 1.9%	99.4% / 0%	1.000	100% / 5.1%	100% / 15.3%
Bézier	1.000	100% / 3.8%	100% / 0%	1.000	100% / 4.6%	98.5% / 0%	0.996	100% / 3.4%	100% / 39.0%
VAE	1.000	100% / 2.5%	100% / 2.1%	1.000	100% / 0.8%	100% / 0%	0.958	100% / 5.1%	100% / 23.7%
Stitch	0.704	2.1% / 5.1%	5.1% / 10.1%	0.423	2.5% / 5.0%	0% / 0%	0.709	0% / 1.7%	0% / 0%

5.4 Diagnostic Analysis

This section analyses the results presented in Sections 5.2 and 5.3 at the feature level.

First, Raw features exhibit substantial cross-domain drift. Table 5.6 compares the median feature values of human and synthetic trajectories across the three games. Human average speed differs by approximately 20-fold between Red Eclipse and CS:GO, decreasing from 1.008 to 0.052 in CS:GO, while `idle_ratio` increases from 0.000 to 0.820, reflecting differences between event-driven and fixed-rate logging. Maximum speed also increases nearly eightfold from Red Eclipse to LoL, from 20.3 to 157.6. Thus, the numerical feature space learned in the source game shifts substantially in the target game, causing the detector to capture game- and logging-specific fingerprints rather than human–bot behavioural differences. In contrast, `avg_turn_angle` varies only from 0.262 to 0.363 across the three games, approximately 1.4-fold compared with the much larger variation in speed-related features. Moreover, Scripted and VAE trajectories exhibit lower turning angles (0.151–0.165) than humans (0.262–0.363) in five of the six game–bot combinations. The only exception is the VAE in CS:GO (0.271). This stable separation supports the finding of Zheng et al. [12] that angle-based measures are insensitive to computing platform differences and provides the empirical basis for the scale-invariant feature design.

Table 5.7 diagnoses the inversion observed for Red Eclipse $\rightarrow$ LoL under Raw features by comparing the direction of the human–bot gap for the most important features in the Red Eclipse model. The inversion results from several interacting feature shifts rather than a single feature. For `dist_std`, the most important feature (importance 0.382), the human–bot gap decreases from +9.58 in Red Eclipse to +3.18 in LoL, weakening the main signal. In contrast, `speed_max` increases sharply from +9.75 to +131.6. More critically, `avg_speed` flips direction: bots are faster than humans in Red Eclipse but slower in LoL. The model therefore applies the learned rule that higher speed indicates bot behaviour in the wrong direction in the target domain. Although less influential, `avg_turn_angle` also reverses its human–bot relationship. These combined changes cause the model’s most important decision cues to shrink, expand, or reverse across domains, producing the observed ranking inversion. Under the scale-invariant features, however, the gap directions of the important features remain consistent between games, explaining why the SI set rescues Bézier.

The VAE fails to transfer reliably under both Raw and SI-min features, with AUCs of 0.384 and 0.414 for Red Eclipse $\rightarrow$ CS:GO and 0.500 and 0.692 for CS:GO $\rightarrow$ Red Eclipse, respectively. However, both directions recover completely under SI-ext. This demonstrates that the failure is not caused by feature scale; otherwise, the SI set would have solved it. Instead, VAE-generated trajectories require finer-grained statistics to expose their cross-game signature. Table 5.8 reports that the SI-ext VAE detector trained on Red Eclipse relies primarily on turning-angle distribution and step-to-step structure: turning-angle standard deviation (`turn_angle_std`, 0.315), step-to-step autocorrelation (`step_autocorr`, 0.277) and mean turning angle (`turn_angle_mean`, 0.179) together account for more than 70% of the feature importance. In contrast, `vh_ratio` (0.014) and `xy_corr` (0.003), which may capture game-specific characteristics, contribute very little. This pattern matches the VAE’s design giveaway: over-smoothed trajectories produce overly regular turning-angle changes and excessive coherence between adjacent steps. Because these properties are expressed as unitless statistics, they transfer across games. Notably, `accel_cv`, which belongs to SI-min, has an importance of only 0.002 in SI-ext. Once micro-structural features are available, the detector largely abandons the coarse acceleration ratio. Hence, the failure of SI-min is explained directly by the features it lacks: turning-angle distribution and step-to-step autocorrelation.

To determine whether the performance gain from SI-ext was driven by game-specific fingerprint features, an ablation test was conducted. First, `xy_corr` was removed from the SI-ext, followed by the removal of `vh_ratio`. The VAE maintained a calibrated detection rate of 100% in all three transfer directions, with only minor changes in AUC. For Red Eclipse $\rightarrow$ LoL, AUC was 0.958, 1.000 and 0.998 for the full SI-ext set, SI-ext without `xy_corr`, and SI-ext without both features, respectively. For Red Eclipse $\rightarrow$ CS:GO, the corresponding values were 1.000, 0.979 and 0.979, while CS:GO $\rightarrow$ Red Eclipse remained 1.000 under all three settings (Table 5.9). Removing the two potentially game-specific features therefore did not reduce performance; AUC

even increased slightly for Red Eclipse $\rightarrow$ LoL. These results confirm that the VAE’s cross-game detection is driven by stable micro-structural features, particularly turning-angle and step-to-step statistics, rather than by game-specific fingerprints.

The results of this chapter can be summarised in relation to the research questions. First, all four synthetic bots are detectable within each of the three games, establishing the validity of the in-domain task and providing a basis for interpreting the cross-game results. A detector trained on one game can detect synthetic bots in another game without retraining, in both within-genre and cross-genre settings, but transfer is unreliable with Raw features. Four scale-invariant ratio features are sufficient to detect rule-based and curve-based attacks across all transfer directions, whereas the learned VAE requires the additional turning-angle and step-to-step statistics in the SI-ext. Thus, attack realism determines the level of feature detail required for transfer, rather than whether detection is possible. All three attacks generated from scratch remain detectable and transferable, whereas the human-derived Stitch bot is substantially more challenging: it requires session-level aggregation for stable in-domain detection and remains undetectable across games. This limitation defines the current scope of the proposed method.

Table 5.6: Feature medians of humans and synthetic bots.

Game	Class	avg_speed	idle_ratio	avg_turn_angle	speed_std	speed_max	dist_std
Red Eclipse	Human	1.008	0	0.331	1.700	20.256	30.242
	Scripted	1.268	0	0.162	0.852	6.872	13.363
	Bézier	1.296	0	0.357	1.226	10.503	20.665
	VAE	0.746	0.04	0.163	1.018	11.876	14.583
	Stitch	1.020	0	0.347	1.768	23.002	32.017
CS:GO	Human	0.052	0.82	0.262	0.137	1.903	2.273
	Scripted	0.131	0.235	0.163	0.088	0.415	1.400
	Bézier	0.135	0.331	0.358	0.127	0.911	2.037
	VAE	0.027	0.901	0.271	0.054	0.854	0.861
	Stitch	0.052	0.818	0.273	0.140	1.908	2.307
LoL	Human	2.559	0.01	0.363	3.161	157.587	7.144
	Scripted	2.119	0	0.165	1.407	12.207	2.454
	Bézier	2.504	0	0.325	2.059	26.000	3.961
	VAE	2.439	0.19	0.151	2.526	30.436	2.750
	Stitch	2.764	0.04	0.382	3.810	402.618	8.223

Table 5.7: Bézier Red Eclipse $\rightarrow$ LoL inversion diagnostic.

Feature	Importance	Gap (Red Eclipse)	Gap (LoL)	Sign flip
dist_std	0.382	+9.58	+3.18	no
speed_max	0.199	+9.75	+131.6	no
avg_speed	0.177	-0.29	+0.05	yes
speed_std	0.143	+0.47	+1.10	no
avg_turn_angle	0.099	-0.03	+0.04	yes
idle_ratio	0.000	0	~ 0.01	—

Table 5.8: SI-ext feature importance (VAE, Red Eclipse).

Feature	Importance
turn_angle_std	0.315
step_autocorr	0.277
turn_angle_mean	0.179
speed_cv	0.145
efficiency_sub	0.042
speed_peak	0.023
vh_ratio	0.014
xy_corr	0.003
accel_cv	0.002
dir_change_rate	0.000

Table 5.9: VAE ablation on SI-ext: AUC / calibrated detection rate.

Feature set	Red Eclipse $\rightarrow$ LoL	Red Eclipse $\rightarrow$ CS:GO	CS:GO $\rightarrow$ Red Eclipse
SI-ext (all 10)	0.958 / 100%	1.000 / 100%	1.000 / 100%
without xy_corr	1.000 / 100%	0.979 / 100%	1.000 / 100%
without xy_corr, vh_ratio	0.998 / 100%	0.979 / 100%	1.000 / 100%

Discussion, Limitations and Future Work

6.1 Principal Findings

The results of Chapter 5 lead to several core findings.

First, a shared human–machine behavioural signal exists across games, and it can be detected without a deep model when represented in a scale-invariant representation. The contrast between the raw and ratio-based feature sets demonstrates that cross-game failure does not arise because the behavioural signal is absent, but because the representation is inappropriate. Raw features enable the detector to learn game- and logger-specific fingerprints, including differences in units, sampling mechanisms and mouse sensitivity, which do not transfer to a new game. When these absolute values are replaced by ratios and angles, the game-specific scale is largely removed and the underlying behavioural signal becomes transferable. Thus, what transfers across games is not the absolute feature values but the direction and relative structure of the human–machine difference: bots exhibit less turning and more consistent speed, and these relationships remain sufficiently stable across games despite substantial differences in units and logging mechanisms.

Next, attack realism determines the granularity of the features required for detection, rather than whether detection is possible. The answer to RQ2 is therefore more nuanced than the simple claim that more human-like attacks are harder to detect. Mechanical attacks produce obvious behavioural artefacts that can be captured with a small number of features. More realistic generators yield subtler artefacts, requiring richer feature representations to detect them across games. Attacks derived entirely from human segments leave little artificial structure for the detector to learn. The experimental results of the four bots follow this progression exactly.

Scripted and Bézier transfer across all three directions using only four ratio features. The VAE fails with the four-dimensional set but recovers fully with the ten-dimensional set because its over-smoothed signature remains detectable in finer micro-structure. Stitch, by contrast, does not transfer under any setting. Thus, as attack realism increases, the defending system requires finer scale-invariant features rather than a larger model. The ordering of difficulty exactly matches the realism ordering of the generation methods in Section 3.3.

In addition, Stitch cannot be detected in any transfer direction, revealing a blind spot of kinematics-based detection for replay-style attacks. Its content is human movement, and its only giveaway (the seams) is both weak and not scale-invariant. ReMouse [23] addresses this type of attack using similarity matching, relying on excessive similarity between an incoming trajectory and an existing record rather than on behavioural kinematics. These findings therefore support a layered anti-cheat strategy: kinematic detection can target attacks generated from scratch, while similarity- or repetition-based detection can intercept replay attacks. The two approaches are complementary rather than interchangeable.

These findings have direct implications for anti-cheat deployment when a new game is launched.

First, zero-shot cross-game detection is feasible for common attack forms, particularly scripted and curve-based humanisation attacks (see Section 2.3). A detector trained on an existing game can identify these attacks in a new game without attack samples from the target environment, adopting only unlabelled human data for threshold calibration.

Next, FPS-to-MOBA transfer performs no worse than FPS-to-FPS transfer, indicating that transferability depends more on whether the bot’s behavioural signature is game-specific than on genre similarity. As attacks become more realistic, defenders should prioritise scale-invariant micro-structural features, such as step-to-step autocorrelation and turning-angle distributions, rather than simply increasing model capacity.

Finally, detection should operate at multiple temporal scales. Mechanical attacks can be detected from short 10-second windows, whereas replay-type attacks require session-level aggregation; for example, CS:GO Stitch detection increases from 61.6% at the window level to 100% after session-level aggregation. In practice, both detection granularities should operate concurrently to maximise coverage.

6.2 Limitations

The limitations below are ordered by their potential impact on the conclusions, with each defining the conditions under which the findings should be interpreted with caution.

A primary limitation is that the attacks are synthetic proxies rather than real-world cheats. The four bots here represent the standard attack families discussed in the literature (Section 2.3) and are

calibrated to human statistics from their respective games, but they are not trajectories collected from actual cheat programs. The findings therefore establish detection and transferability for four synthetic generation families rather than validated performance against cheats observed in the wild. Nevertheless, synthetic generation is standard practice in the literature, and the four families correspond to scripted cheats, humanisation tools, AI-generated imitation and replay attacks. Real attacks that fall within this spectrum may therefore be expected to exhibit similar levels of detectability.

A further limitation concerns the number of players, particularly in LoL, which contains only seven players and 59 trajectories. Its large fold-to-fold variation means window-level detection rates should be interpreted as feasibility evidence rather than performance estimates, although the AUC and overall trends are more robust. LoL is used only as a target in cross-game evaluation and never as a training source; therefore, its cross-genre results should likewise be interpreted as feasibility evidence. The conclusions for Red Eclipse (45 players) and CS:GO (256 players) are supported by larger samples.

Another consideration is that the datasets were repurposed from studies with different objectives. None was collected specifically for bot detection, and their signal representations differ across device displacement, screen coordinates and view-angle proxies. Environmental factors such as mouse sensitivity and hardware could also not be fully controlled. In CS:GO, for example, participants could adjust mouse sensitivity according to personal preference [27], introducing additional scaling differences into absolute speed features. The scale-invariant feature design was specifically intended to accommodate such heterogeneous environments, and its successful cross-game performance demonstrates its robustness under these conditions.

The study is also limited by its single-modality design. It relies exclusively on mouse movement and does not incorporate keyboard dynamics or in-game behavioural events, including kills and movement commands. Multimodal approaches may provide additional discriminatory information; for example, Chu et al. [16] combined keyboard and mouse dynamics.

Finally, the synthetic attacks model only the kinematic layer of cheating. They reproduce statistical properties of mouse movement but do not model task-level behaviour, such as where a player aims, when movement stops, or when long pauses occur during gameplay. Consequently, even the most realistic synthetic attacks remain somewhat removed from the full behavioural complexity of real-world cheats.

6.3 Future Work

The limitations also indicate several directions of future research. The following directions correspond to the limitations discussed above, with a final methodological extension.

A main direction is validation against real cheat telemetry. Once a game dataset with cheat labels appears, the evaluation protocol of this study (features, calibration, leakage-safe splitting) can be applied directly to determine how well the conclusions based on synthetic proxies generalise to real-world attacks.

A further direction is to expand the range of games and genres. Adding more FPS titles and other genres, such as real-time strategy (RTS) and MMORPGs, would test whether this study’s conclusions about the realism spectrum hold generally across genres.

Another direction is to incorporate keyboard and multimodal signals. Combining keyboard and mouse is an established route in the literature (Chu et al. [16] use both). For attacks such as Stitch, which leave no giveaway at the mouse layer, cross-modal inconsistency may offer an additional detection signal.

The generation framework could also be extended to task-aware generation. Conditioning the generator on game state (target positions, combat events) would introduce task structure and contextual pauses into the generated trajectories, allowing the detector to be evaluated against attacks capable of reproducing aspects of actual gameplay.

Finally, the methodology itself can be extended. Incorporating more generation types (GAN, diffusion) would complete the spectrum, while evaluating alternative classifiers would test whether the conclusions are robust to model choice.

CHAPTER 7

Conclusion

This study systematically evaluated whether behaviour-based bot detectors can transfer across games. Three public game datasets (two FPS, one MOBA) were unified into the (dx, dy, t) trajectory format and tested against four synthetic attacks of increasing realism (rule-based, parametric-curve, learned, human-derived). Detectors were evaluated in three zero-shot transfer directions using player-disjoint protocols and thresholds calibrated solely from unlabelled human data in the target game.

The results demonstrate that a transferable human–machine behavioural signal exists, but it is reliably captured only via scale-invariant features. Raw features produced unpredictable transfer, with only four of twelve combinations usable and failures arising from three mechanisms: ranking inversion, threshold collapse and signal loss. In contrast, the ten-dimensional scale-invariant feature set achieved 100% calibrated detection for all attacks generated from scratch across all three transfer directions, with $AUC \geq 0.958$ and false-positive rates $\leq 5.1\%$. Attack realism determined the required feature resolution rather than whether transfer was possible: rule-based and parametric-curve attacks were detected with only four features, whereas the learned VAE attack required the extended micro-structure features. The human-derived Stitch attack remained undetectable across games, defining the current boundary of the method.

Overall, zero-shot cross-game bot detection is feasible for attacks generated from scratch when scale-invariant representations are combined with target-domain calibration using only unlabelled human data. This provides a practical cold-start strategy for anti-cheat deployment: detectors trained on existing games can immediately address script-level attacks in a recently launched game, while additional features and data can be incorporated as the new environment matures. Future work should extend this framework to real cheat telemetry, multimodal behavioural signals and task-aware synthetic attacks.

References

- [1] S. Gianvecchio, Z. Wu, M. Xie, and H. Wang, “Battle of Botcraft: Fighting bots in online games with human observational proofs,” in *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS '09)*. ACM, 2009, pp. 256–268.
- [2] A. Kanervisto, T. Kinnunen, and V. Hautamäki, “GAN-Aimbots: Using machine learning for cheating in first person shooters,” *IEEE Transactions on Games*, vol. 15, no. 4, pp. 566–579, 2023.
- [3] S. Khan, C. Devlen, M. Manno, and D. Hou, “Mouse dynamics behavioral biometrics: A survey,” *ACM Computing Surveys*, vol. 56, no. 6, pp. 1–33, 2024.
- [4] J. P. Pinto, A. Pimenta, and P. Novais, “Deep learning and multivariate time series for cheat detection in video games,” *Machine Learning*, vol. 110, no. 9, pp. 3037–3057, 2021.
- [5] D. Buckley, K. Chen, and J. Knowles, “Rapid skill capture in a first-person shooter,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 9, no. 1, pp. 63–75, 2017.
- [6] M. Pusara and C. E. Brodley, “User re-authentication via mouse movements,” in *Proceedings of the 2004 ACM Workshop on Visualization and Data Mining for Computer Security (VizSEC/DMSEC '04)*. ACM, 2004, pp. 1–8.
- [7] H. Gamboa and A. Fred, “A behavioural biometric system based on human computer interaction,” in *Proceedings of SPIE*, vol. 5404, 2004, pp. 381–392.
- [8] A. A. E. Ahmed and I. Traore, “A new biometric technology based on mouse dynamics,” *IEEE Transactions on Dependable and Secure Computing*, vol. 4, no. 3, pp. 165–179, 2007.

- [9] P. M. Fitts, “The information capacity of the human motor system in controlling the amplitude of movement,” *Journal of Experimental Psychology*, vol. 47, no. 6, pp. 381–391, 1954.
- [10] T. Flash and N. Hogan, “The coordination of arm movements: An experimentally confirmed mathematical model,” *Journal of Neuroscience*, vol. 5, pp. 1688–1703, 1985.
- [11] A. Acien, A. Morales, J. Fierrez, and R. Vera-Rodriguez, “BeCAPTCHA-Mouse: Synthetic mouse trajectories and improved bot detection,” *Pattern Recognition*, vol. 127, p. 108643, 2022.
- [12] N. Zheng, A. Paloski, and H. Wang, “An efficient user verification system via mouse movements,” in *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS ’11)*. ACM, 2011, pp. 139–150.
- [13] C. Shen, Z. Cai, and X. Guan, “Continuous authentication for mouse dynamics: A pattern-growth approach,” in *Proceedings of the 42nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2012)*. IEEE, 2012, pp. 1–12.
- [14] C. Feher, Y. Elovici, R. Moskovitch, L. Rokach, and A. Schclar, “User identity verification via mouse dynamics,” *Information Sciences*, vol. 201, pp. 19–36, 2012.
- [15] N. Siddiqui, R. Dave, and N. Seliya, “Continuous user authentication using mouse dynamics, machine learning, and Minecraft,” in *Proceedings of the 2021 International Conference on Electrical, Computer and Energy Technologies (ICECET)*. IEEE, 2021, pp. 1–6.
- [16] Z. Chu, S. Gianvecchio, A. Koehl, H. Wang, and S. Jajodia, “Blog or block: Detecting blog bots through behavioral biometrics,” *Computer Networks*, vol. 57, no. 3, pp. 634–646, 2013.
- [17] C. Iliou, T. Kostoulas, T. Tsikrika, V. Katos, S. Vrochidis, and I. Kompatsiaris, “Detection of advanced web bots by combining web logs with mouse behavioural biometrics,” *Digital Threats: Research and Practice*, vol. 2, no. 3, pp. 1–26, 2021.
- [18] S. Zhao, J. Fang, S. Zhao, R. Wu, J. Tao, S. Li, and G. Pan, “T-Detector: A trajectory based pre-trained model for game bot detection in MMORPGs,” in *Proceedings of the 2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2022, pp. 992–1003.
- [19] E. Lee, J. Woo, H. Kim, A. Mohaisen, and H. K. Kim, “You are a game bot!: Uncovering game bots in MMORPGs via self-similarity in the wild,” in *Proceedings of the 23rd Annual Network and Distributed System Security Symposium (NDSS 2016)*. Internet Society, 2016, pp. 1–15.
- [20] M. Antal and N. Fejér, “Mouse dynamics based user recognition using deep learning,” *Acta Universitatis Sapientiae, Informatica*, vol. 12, no. 1, pp. 39–50, 2020.

- [21] M. Antal, K. Buza, and N. Fejér, “SapiAgent: A bot based on deep learning to generate human-like mouse trajectories,” *IEEE Access*, vol. 9, pp. 124 396–124 408, 2021.
- [22] M. Antal and E. Egyed-Zsigmond, “Intrusion detection using mouse dynamics,” *IET Biometrics*, vol. 8, no. 5, pp. 285–294, 2019.
- [23] S. Sadeghpour and N. Vlajic, “ReMouse dataset: On the efficacy of measuring the similarity of human-generated trajectories for the detection of session-replay bots,” *Journal of Cybersecurity and Privacy*, vol. 3, no. 1, pp. 95–117, 2023.
- [24] C. Shen, Z. Cai, X. Guan, Y. Du, and R. A. Maxion, “User authentication through mouse dynamics,” *IEEE Transactions on Information Forensics and Security*, vol. 8, no. 1, pp. 16–30, 2013.
- [25] P. Chong, Y. Elovici, and A. Binder, “User authentication based on mouse dynamics using deep neural networks: A comprehensive study,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 1086–1101, 2020.
- [26] H. Niu, J. Chen, Z. Zhang, and Z. Cai, “Mouse dynamics based bot detection using sequence learning,” in *Biometric Recognition (LNCS Vol. 12878)*. Springer, 2021, pp. 49–56.
- [27] M. Fanourakis and G. Chanel, “AMuCS: Affective multimodal Counter-Strike video game dataset,” *Scientific Data*, vol. 12, p. 1325, 2025.
- [28] H. Lee, S. Lee, R. Nallapati, Y. Uh, and B. Lee, “Characterizing and quantifying expert input behavior in League of Legends,” in *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI ’24)*. ACM, 2024, pp. 1–17.
- [29] X. Wang, Y. Shi, K. Zheng, Y. Zhang, W. Hong, and S. Cao, “User authentication method based on keystroke dynamics and mouse dynamics with scene-irrelated features in hybrid scenes,” *Sensors*, vol. 22, no. 17, p. 6627, 2022.
- [30] G. Zhang, Z. Hu, M. Bâce, and A. Bulling, “Mouse2Vec: Learning reusable semantic representations of mouse behaviour,” in *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI ’24)*. ACM, 2024, pp. 1–17.
- [31] P. Mavromoustakos-Blom, S. Bakkes, and P. Spronck, “Towards multi-modal stress response modelling in competitive League of Legends,” in *Proceedings of the 2019 IEEE Conference on Games (CoG)*. IEEE, 2019, pp. 1–4.

GitLab Repository

The project repository is available at:

<https://git.cs.bham.ac.uk/projects-2025-26/yxc1228>

Repository contents

The repository contains the full implementation of this project. The `src` directory holds the Python modules: data loaders for the three games, feature extraction, the four synthetic bot generators, and the evaluation and diagnostic code. The `notebooks` directory contains four notebooks, one per game for data processing and in-domain evaluation, and one for the cross-game experiments. The `artifacts` directory stores cached feature tables, trained classifiers and versioned VAE weights, so that the main results can be reproduced without the raw datasets. A small sample of raw data is included for testing the loaders; the full datasets are externally licensed and are therefore not committed. Setup instructions and further details are documented in the repository README.

Running the code

The main cross-game results can be reproduced by running `notebooks/cross_game.ipynb`, which reads the committed artifacts and requires no raw data. A full rebuild from raw data runs the three per-game notebooks first, followed by the cross-game notebook; the required datasets and the exact order are described in the README.

Supplementary Figures

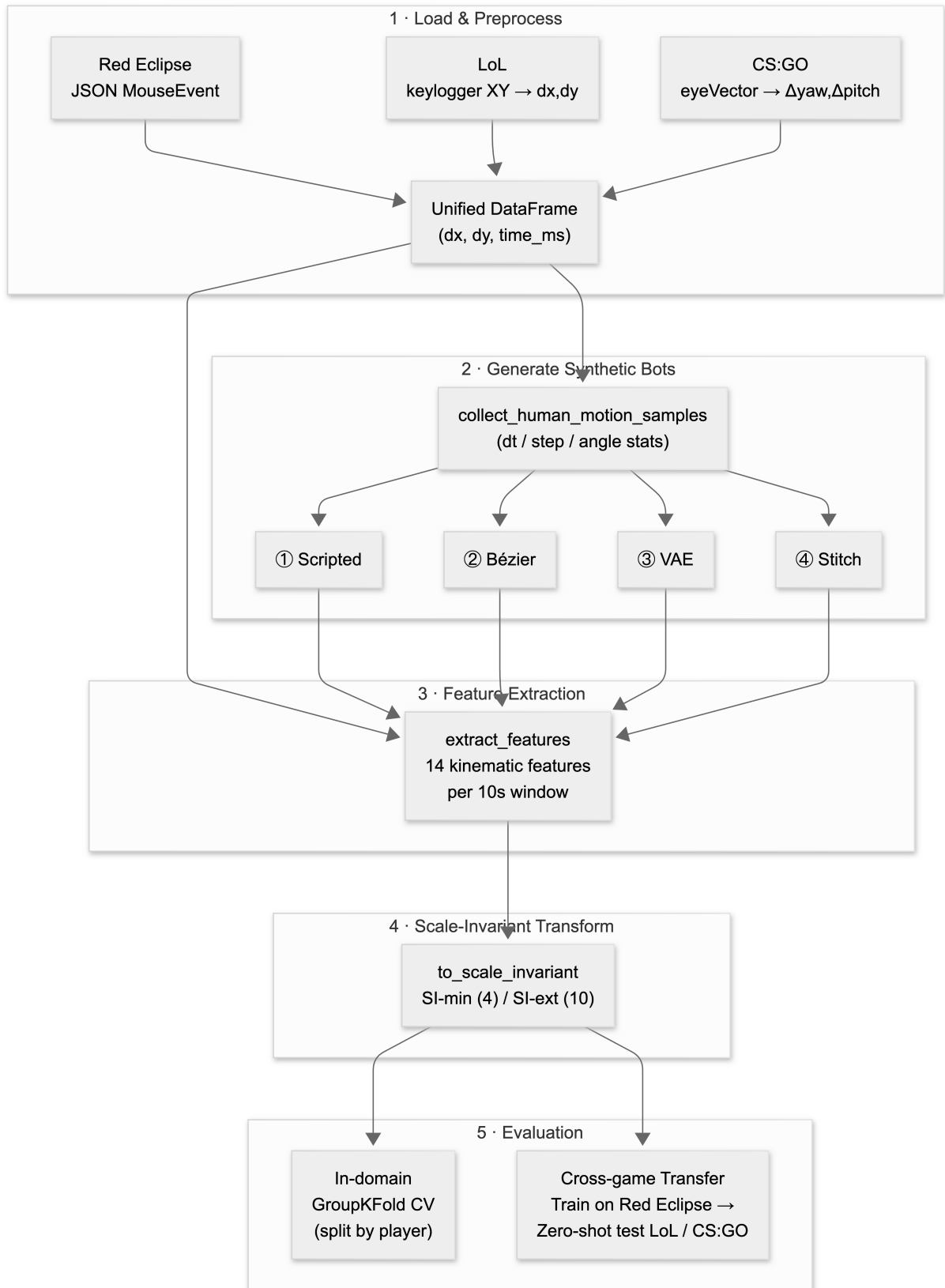


Figure 1: Processing pipeline from raw game data to evaluation.

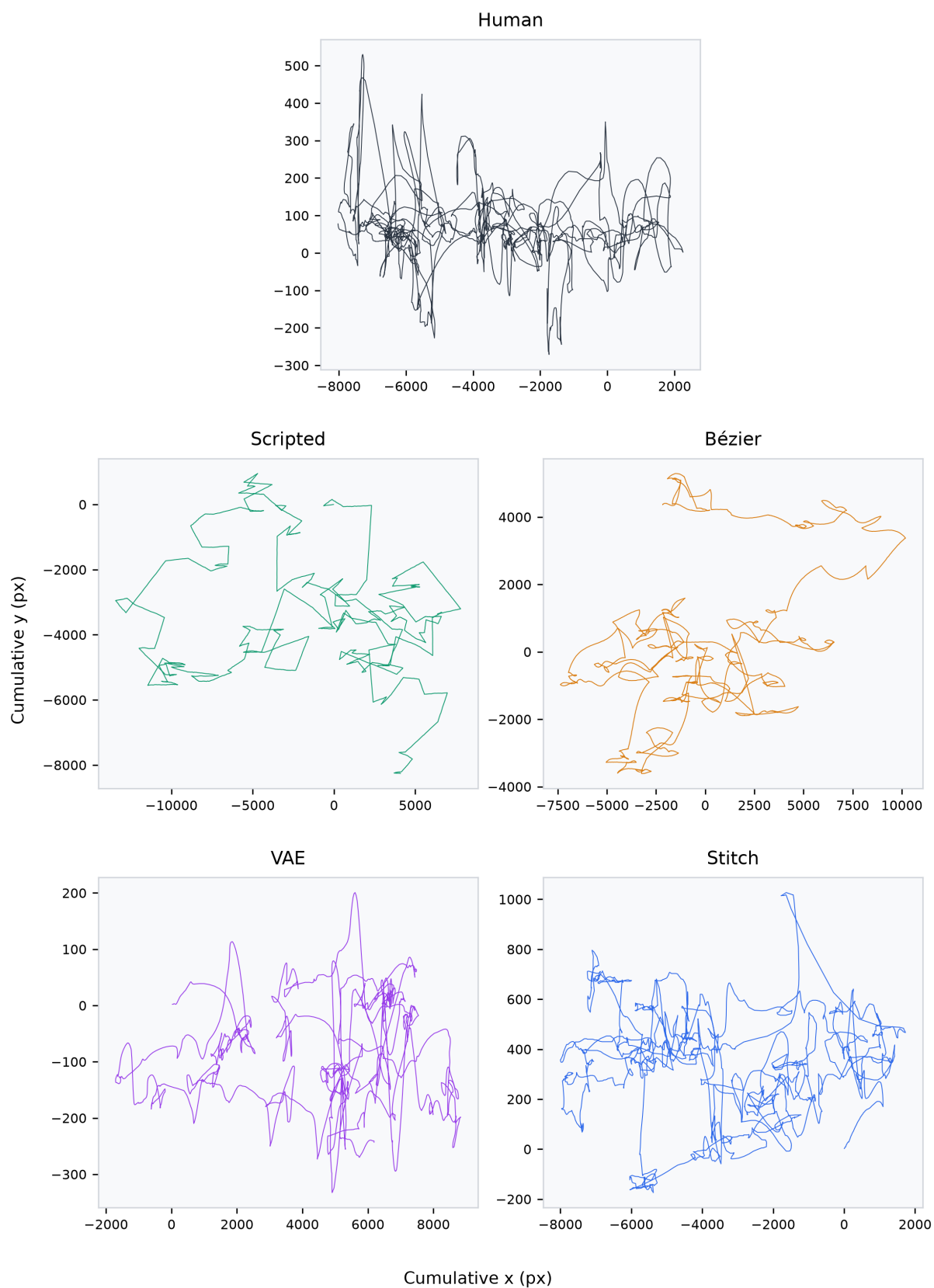


Figure 2: Cumulative mouse paths over one three-minute Red Eclipse session for a human player and for each generator. Axes are scaled independently per panel.

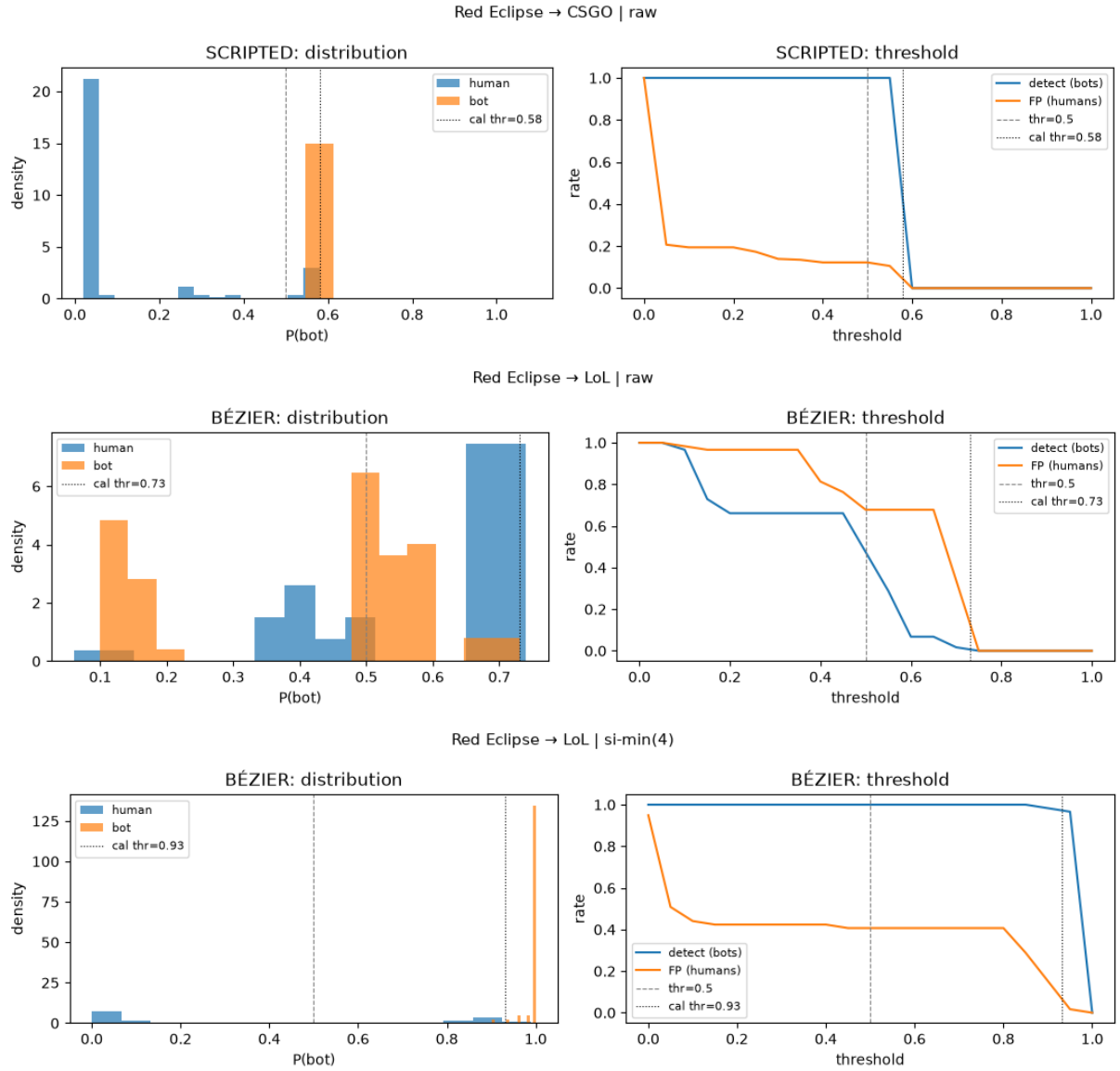


Figure 3: Detector score distributions and threshold sweeps for three cross-game cases. From top: Scripted Red Eclipse to CS:GO with Raw features, Bézier Red Eclipse to LoL with Raw features, and the same Bézier case with SI-min.

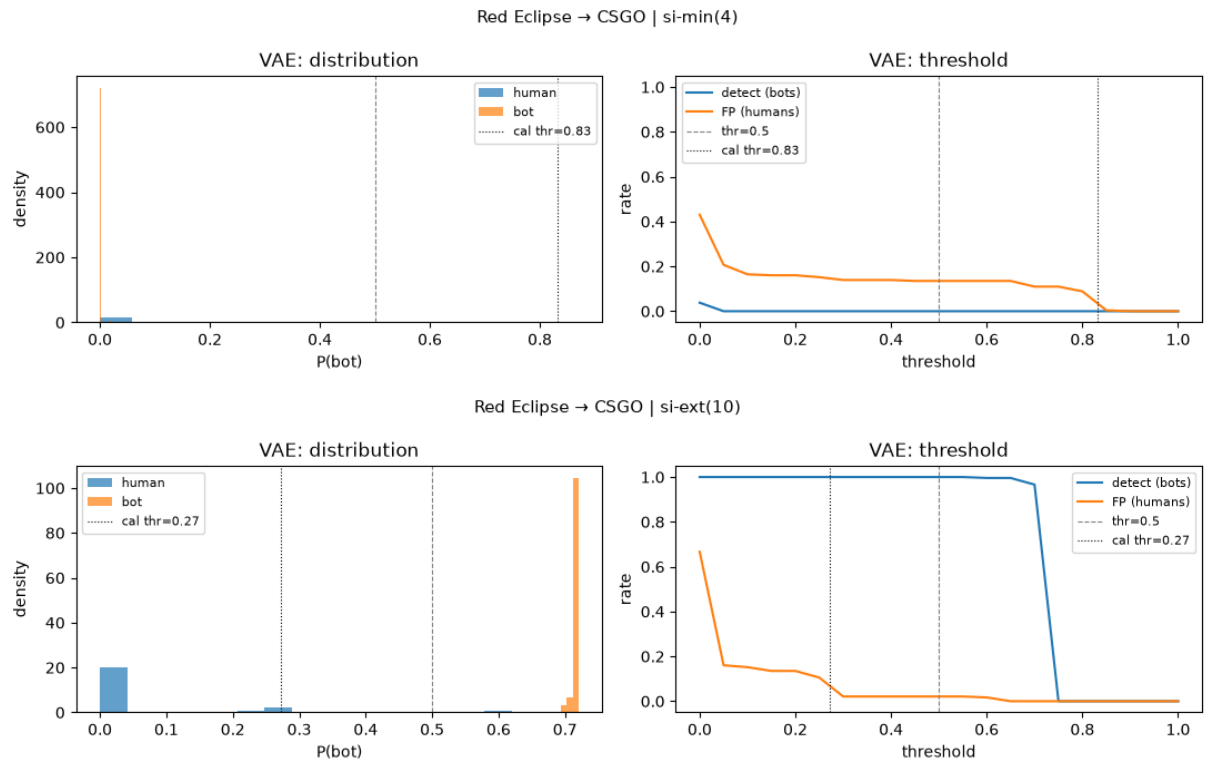


Figure 4: Detector score distributions and threshold sweeps for VAE Red Eclipse to CS:GO with SI-min (top) and SI-ext (bottom).